Programming Project #4
EGRE246
Text Cross Referencing

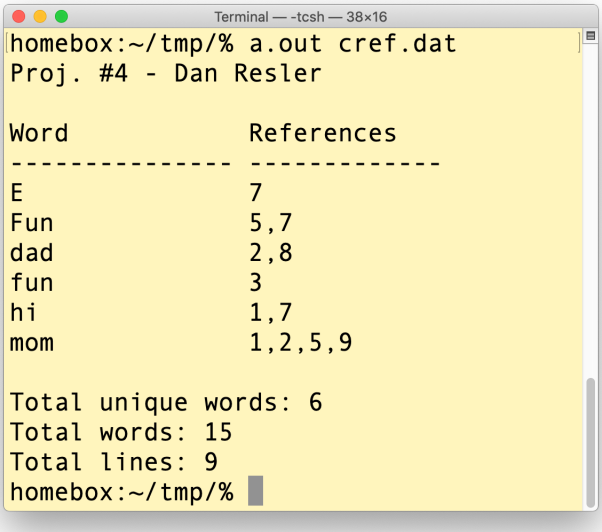
1 Overview

Write a C program that inputs a paragraph of text from a file given on the command line and then lists all the words in the text in alphabetical order along with the line numbers (where the first line is line #1) of all their occurrences plus some stats. For example, the text file (file `cref.dat`, available on Canvas)

```
hi hi hi mom
mom         dad
fun

Fun mom-Fun
. ? -
E hi! Fun???!!!@
dad  -13
        mom
```

would produce the following sample run:



Note that spaces, punctuation, and all numbers are ignored. Use the ordering defined by `strcmp` from the C library (i.e. uppercase < lowercase). Your program should be able to handle up to 10,000 unique words.

Your program should produce an error message and exit should an non-existent file be specified on the command line. Your code should not use any global variables (exception: preprocessor values) and you should be sure to subdivide your code into appropriate function calls. You may work in

pairs for this project, and if you do you should only turn in a single copy of the project as a single file.

2 Implementation Requirements & Helps

1. Do not read in and store the entire text file! Instead read in a complete single line at a time (assume a maximum size of 200 characters) and then parse the line into individual strings, storing them as you go; more on this later.
2. **Make sure you do not code up any of the string processing until you have the input working correctly.**
3. Recommended implementation steps:
   - get file input working
   - implement dividing up the input into strings and putting them in a table
   - work on creating your list of line numbers for each word
   - sort the table
   - print out the table
4. Probably the easiest way to input a whole line of text is by using `fgets` from the C library. Note however that `fgets` leaves the newline in the inputted string!
5. After you input a whole line of text you will need to 'chop' it into individual strings for processing. I recommend you use `strtok` from the C library – **this will save you a lot of time and grief!**
6. You *must* use the singly-linked list module `llist` to create and maintain your list of line numbers for each word. You can use the codes in Lab7 and Lab8 for the implementation of the singly-linked list module.
7. If you require to sort your list of words (it's not the only way to establish an ordering) I recommend you use `qsort` from the C library.
8. Your output should be formatted and presented exactly as in the sample run above. Note that the word field is 15 spaces wide – do not worry if any strings exceed this number.
9. Any recommendations above are exactly that – you are not required to follow them in your code.

3 Deliverables

Submit your single file solution via Canvas in the usual fashion. You may work in pairs on this project; submit only a single copy making sure that both of your names are on the file if you work with someone else. Be sure to document your code in the manner previously discussed in class.